



COM Practice

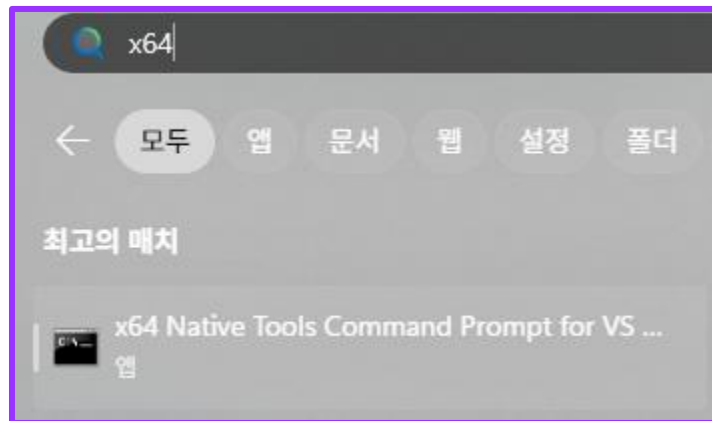
contents

- COM Practice
 - MathServer.dll, client.exe → x64
 - Rewind diff
 - COM example
 - Edit mathserver.cpp
 - Write com_mymath.py
 - Run
 - diff. IUnknown, IDispatch
 - Edit mathserver.cpp
 - Run v2.0
 - Issue

1. MathServer.dll → x64로 생성 및 레지스트리 등록

MathServer.dll, client.exe → x64

- Shit !!!.. x64 Native Tools Command Prompt for VS 2022



Rewind diff

- diff. In-Proc, Out-of-Proc

변경 대상	In-Proc	Out-of-Proc	비고
서버 형태	DLL	EXE	COM 객체가 DLL인지, EXE인지
COM 활성화 방식	DLL 직접 로드	EXE 서버 실행 후 연결	COM Runtime이 DLL을 직접 로드하는지, EXE 서버를 실행한 뒤 IPC로 연결하는지의 차이
Registry 등록 방식	InprocServer32	LocalServer32	COM이 서버 위치를 찾기 위해 사용하는 Registry Key가 다름
COM Factory 등록 방식	DllGetClassObject()	CoRegisterClassObject()	DLL은 COM Runtime이 DllGetClassObject()를 직접 호출해 Factory 획득, EXE는 서버가 스스로 생성 가능함을 COM Runtime에 등록 필요
프로세스 유지 방식	클라이언트 종속	자체 실행 유지 필요	EXE 서버는 계속 실행 상태를 유지해야 하므로 메시지 루프 필요
CoCreateInstance() > flag	CLSCTX_INPROC_SERVER	CLSCTX_LOCAL_SERVER	객체를 현재 프로세스 내부에서 생성할지, 외부 프로세스에서 생성할지 지정
호출 방식	직접 함수 호출	RPC / IPC 기반 호출	EXE 방식은 Proxy/Stub를 통한 Marshalling 발생

Rewind diff

- Step 2. 뭘 바꿔야 하는가?

- ✓ In-Proc Server 구조

- MathServer.dll
- DllGetClassObject > Factory 생성 > Server 생성
- Reg Key: InprocServer32

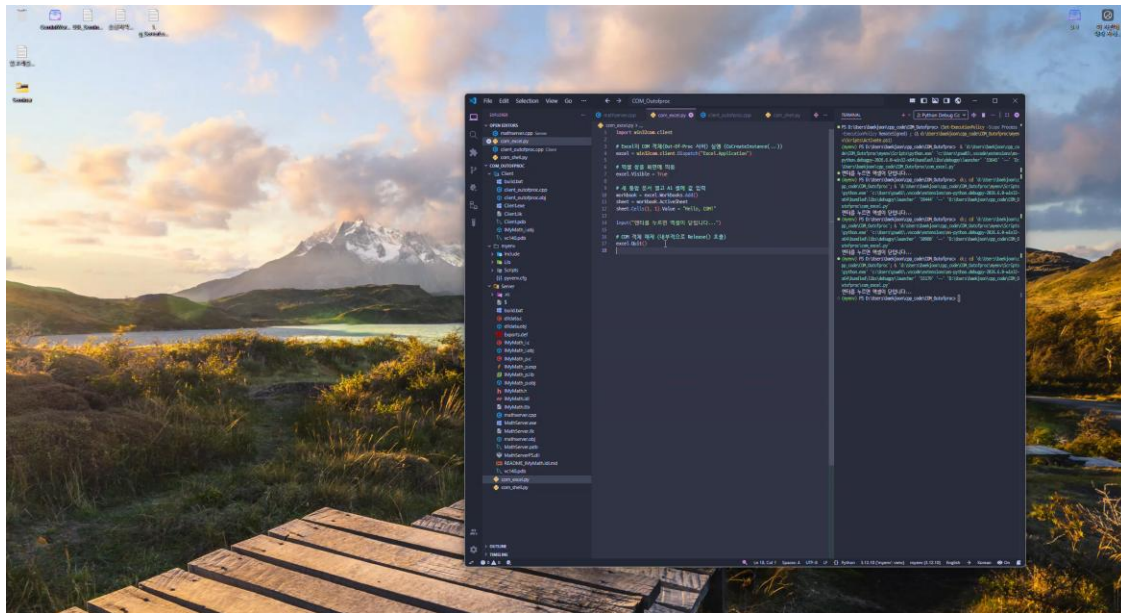
- ✓ Out-of-Proc Server 구조

- MathServer.exe (Factory 생성 > Server 생성 흐름은 동일)
- COM Runtime이 EXE와 Client 사이를 RPC(Remote Procedure Call)로 연결
- EXE가 COM Server 역할 수행
- Reg Key: LocalServer32
- 서버와 IPC/RPC 연결
→ Proxy/Stub: MathServerPS.dll 필요
- 주요 함수 변동
 - DllMain (DLL Entry)
 - DllGetClassObject
 - DllCanUnloadNow
 - DllRegisterServer
 - DllUnregisterServer
 - CoRegisterClassObject() // Factory 개체를 OLE에 등록해 다른 App이 연결 가능하게 함
 - CoResumeClassObjects() // 등록 후 COM Runtime에 "나 준비 완료" 알림
 - CoRevokeClassObject() // 프로그램 종료 시 등록된 Factory 해제
 - main()

COM Example

- Excel COM Example

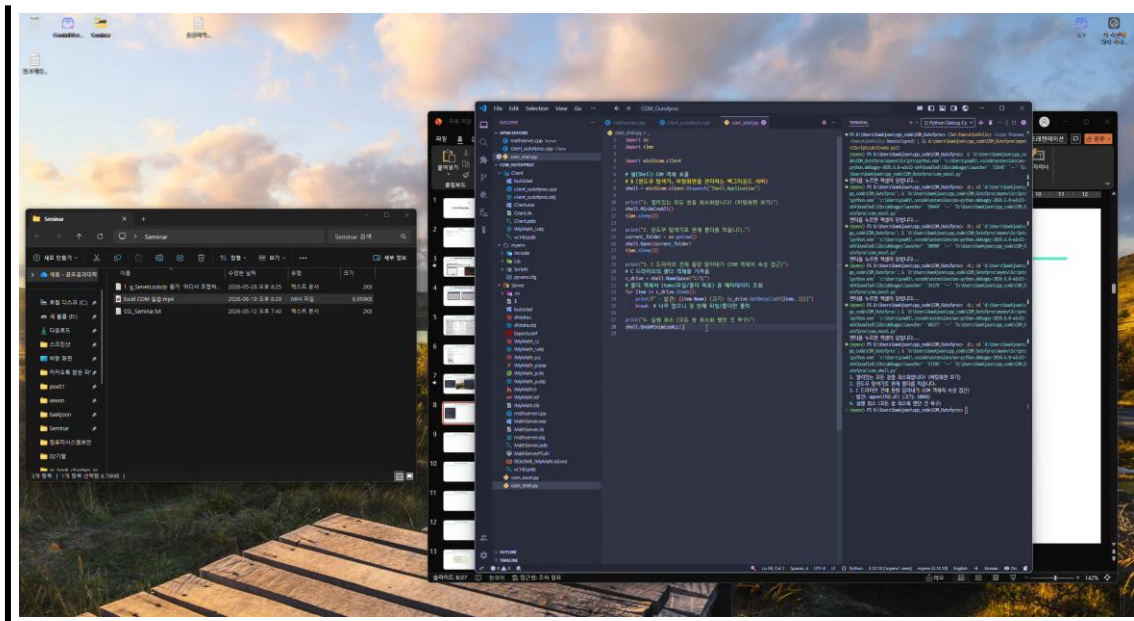
```
com_excel.py > ...
1 import win32com.client
2
3 # Excel의 COM 객체(Out-Of-Proc 서버) 실행 (CoCreateInstance(...))
4 excel = win32com.client.Dispatch("Excel.Application")
5
6 # 엑셀 창을 화면에 띄움
7 excel.Visible = True
8
9 # 새 통합 문서 열고 A1 셀에 값 입력
10 workbook = excel.Workbooks.Add()
11 sheet = workbook.ActiveSheet
12 sheet.Cells(1, 1).Value = "Hello, COM!"
13
14 input("엔터를 누르면 엑셀이 닫힙니다...")
15
16 # COM 객체 해제 (내부적으로 Release() 호출)
17 excel.Quit()
```



COM Example

- Shell COM Example

```
com_shell.py > ...
1 import os
2 import time
3
4 import win32com.client
5
6 # 열(Shell) COM 객체 호출
7 # $ (윈도우 탐색기, 바탕화면을 관리하는 백그라운드 서버)
8 shell = win32com.client.Dispatch("Shell.Application")
9
10 print("1. 열려있는 모든 창을 최소화합니다! (바탕화면 보기)")
11 shell.MinimizeAll()
12 time.sleep(2)
13
14 print("2. 윈도우 탐색기로 현재 폴더를 띄웁니다.")
15 current_folder = os.getcwd()
16 shell.Open(current_folder)
17 time.sleep(2)
18
19 print("3. C 드라이브의 전체 용량 알아내기 (COM 객체의 속성 접근)")
20 # C 드라이브의 폴더 객체를 가져옴
21 c_drive = shell.NameSpace("C:\\")
22 # 폴더 객체의 Items(파일/폴더 목록) 중 메타데이터 조회
23 for item in c_drive.Items():
24     print(f"- 발견: {item.Name} (크기: {c_drive.GetDetailsOf(item, 1)})")
25     break # 너무 많으니 첫 번째 파일/폴더만 출력
26
27 print("4. 실행 취소 (모든 창 최소화 했던 것 복구)")
28 shell.UndoMinimizeAll()
```

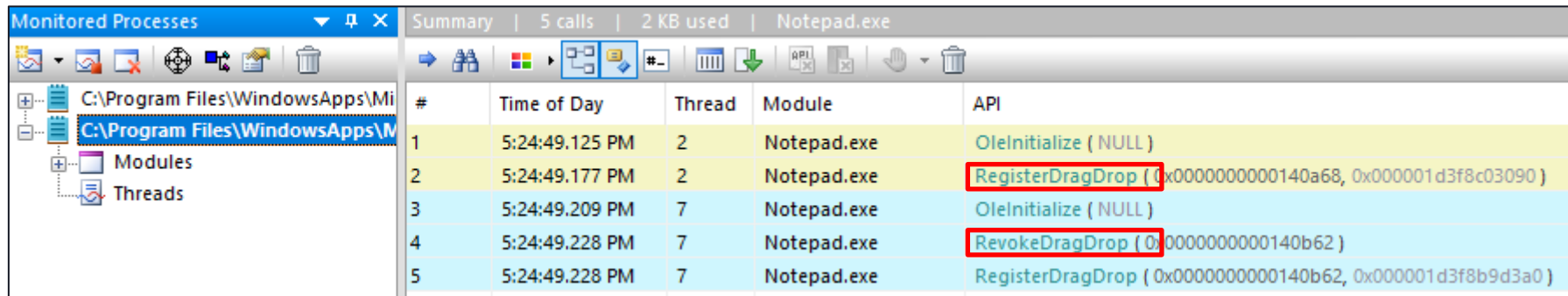


COM Example

- 파일의 Drag & Drop 도 COM이 동작
- 텍스트는 그냥 Win API

```
C:\Program Files\Microsoft Visual Studio\2022\Community>dumpbin /exports C:\Windows\System32\ole32.dll | findstr "Drag"
1012 B5 0000F2C0 DoDragDrop
1013 B6 000094390 DragDropSetEDT
1291 1DB 0000DD70 RegisterDragDrop
1294 1DE 0000E9C0 RevokeDragDrop

C:\Program Files\Microsoft Visual Studio\2022\Community>dumpbin /exports C:\Windows\System32\ole32.dll | findstr "Clip"
809 20 00008B50 ClipboardProcessUninitialize
1240 1A6 00026B40 OleFlushClipboard
1242 1A8 0000E450 OleGetClipboard
1243 1A9 00007F50 OleGetClipboardWithEnterpriseInfo
802 1AC 00008A790 OleGetPackageClipboardOwner
1249 1B0 0003C450 OleIsCurrentClipboard
1271 1C6 000266D0 OleSetClipboard
```

Monitored Processes					
Summary 5 calls 2 KB used Notepad.exe					
					
C:\Program Files\WindowsApps\Mi C:\Program Files\WindowsApps\M					
Modules					
Threads					
#	Time of Day	Thread	Module	API	
1	5:24:49.125 PM	2	Notepad.exe	OleInitialize (NULL)	
2	5:24:49.177 PM	2	Notepad.exe	RegisterDragDrop (0x00000000000140a68, 0x000001d3f8c03090)	
3	5:24:49.209 PM	7	Notepad.exe	OleInitialize (NULL)	
4	5:24:49.228 PM	7	Notepad.exe	RevokeDragDrop (0x00000000000140b62)	
5	5:24:49.228 PM	7	Notepad.exe	RegisterDragDrop (0x00000000000140b62, 0x000001d3f8b9d3a0)	

Edit mathserver.cpp v2.0

- Step 3. mathserver.cpp 변동 (main 함수)

1. CoRegisterClassObject
2. CoResumeClassObject
3. GetMessage
4. CoRevokeClassObject
5. CoUninitialize

```
std::cout << "==== Out-of-Proc COM Server Start ==== " << std::endl;

HRESULT hr = CoInitialize(NULL);
if (FAILED(hr)) {
    std::cout << "[!] COM Initialize Failed." << std::endl;
    return 1;
}

MyMathFactory *factory = new MyMathFactory();

DWORD regid;

/*
MS: EXE 개체를 OLE에 등록해 다른 App이 연결할 수 있도록 합니다.
// - 이 EXE가 CLSID_MyMathComponent 담당 서버임을 COM Runtime에 등록
// - i.e.) DllGetClassObject() 대체
        - REGCLS_MULTIPLEUSE: 여러 프로그램이 하나의 factory를 공유
        - REGCLS_SUSPENDED: 대기 상태로 등록(CoResumeClassObject 호출)
*/
HRESULT hrRegister = CoRegisterClassObject(
    CLSID_MyMathComponent,
    factory,
    CLSCTX_LOCAL_SERVER,
    REGCLS_MULTIPLEUSE | REGCLS_SUSPENDED,
    &regid);
if (FAILED(hrRegister)) {
    std::cout << "[!] CoRegisterClassObject Failed." << std::endl;
    return 1;
}

// 등록 후 COM Runtime에 "나 준비 완료" 신호
HRESULT hrResume = CoResumeClassObjects();
if (FAILED(hrResume)) {
    std::cout << "[!] CoResumeClassObject Failed." << std::endl;
    return 1;
}

// 메시지 루프
int i = 0;
MSG msg;
while (GetMessage(&msg, NULL, 0, 0)) {
    std::cout << "[i] Server received " << ++i << std::endl;
    TranslateMessage(&msg);
    DispatchMessage(&msg);
}

// 프로그램 종료 시 COM Runtime에 등록된 Factory 해제
CoRevokeClassObject(regid);

factory->Release();
CoUninitialize();

return 0;
```

Write com_mymath.py

- **GetModule**: .tlb 파일을 읽고 파이썬 클래스로 변환
- ✓ .tlb: midl이 만들어줌(.idl을 보고)

```
// Generated .IDL file (by the OLE/COM Object Viewer)
//
// typelib filename: <could not determine filename>
[
    uuid(B2C3D4E5-F6A1-4B2C-9D0E-123456789ABC),
    version(1.0),
    custom(DE77BA64-517C-11D1-A2DA-0000F8773CE9, 134283892),
    custom(DE77BA63-517C-11D1-A2DA-0000F8773CE9, 2147483647),
    custom(DE77BA65-517C-11D1-A2DA-0000F8773CE9, "Created by MIDL version 8.0")
]
library MyMathLib
{
    // TLib : OLE Automation : {00020430-0000-0000-C000-000000000046}
    importlib("stdole2.tlb");

    // Forward declare all types defined in this typelib
    interface IMyMath;

    [
        odl,
        uuid(A1B2C3D4-E5F6-4A1B-8C9D-0123456789AB)
    ]
    interface IMyMath : IUnknown {
        HRESULT _stdcall AddPoint(
            [in] tagPOINT3D* p,
            [out, retval] int* result);

        HRESULT _stdcall Add(
            [in] int a,
            [in] int b,
            [out, retval] int* result);

        HRESULT _stdcall Subtract(
            [in] int a,
            [in] int b,
            [out, retval] int* result);

        HRESULT _stdcall Multiply(
            [in] int a,
            [in] int b,
            [out, retval] int* result);

    };

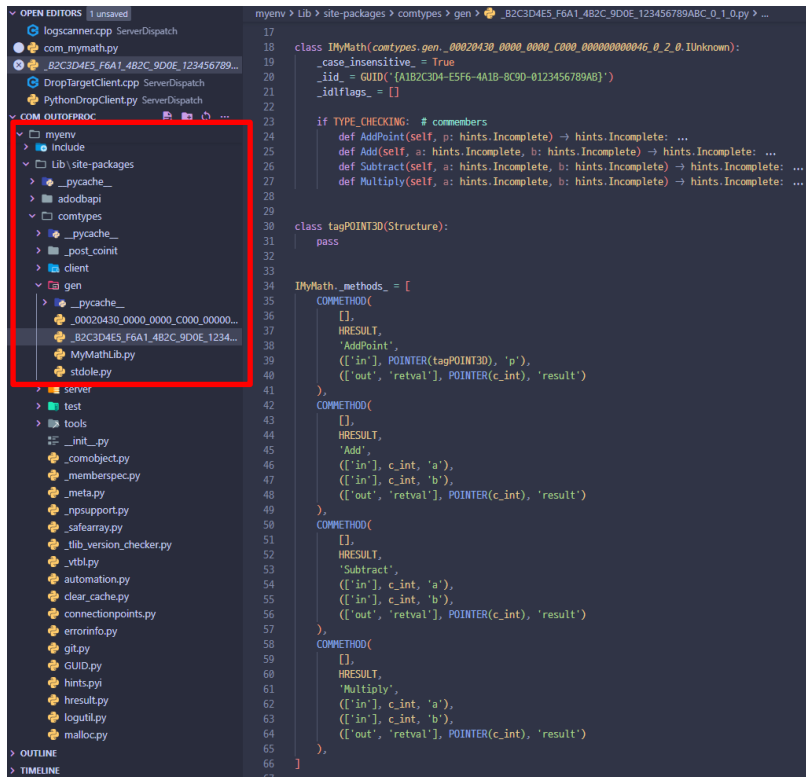
    typedef struct tagtagPOINT3D {
        int x;
        int y;
        int z;
    } tagPOINT3D;

    [
        uuid(08564873-A821-4119-B5EE-79DCA34324A5)
    ]
    coclass MyMathComponent {
        [default] interface IMyMath;
    };
};
```

```
com_mymath.py > ...
1  import os
2  from ctypes import byref
3
4  # COM 서버: IUnknown 기반의 C++ 전용 인터페이스 > 파이썬에서 구조체와 포인터를 읽
5  from ctypes import CLSCTX_LOCAL_SERVER
6  from ctypes.client import CreateObject, GetModule
7  from ctypes.gen import MyMathLib
8
9  print("1. 타입 라이브러리(IMyMath.tlb) 분석 중...")
10 # midl이 만들어준 .tlb 파일을 파이썬이 읽어서 알아서 파이썬 클래스로 변환해 줌
11 tlb_path = os.path.abspath(r"Server\IMyMath.tlb")
12 GetModule(tlb_path)
13
14 # 파이썬 코드로 자동 번역된 라이브러리 가져오기
15
16 print("2. MathServer.exe (Out-Of-Proc) 실행 및 연결!")
17 # CoCreateInstance(CLSID_MyMathComponent...)를 내부 호출
18 math_server = CreateObject(MyMathLib.MyMathComponent, clsctx=CLSCTX_LOCAL_SERVER)
19
20 print("3. 파이썬에서 사용자님의 C++ 함수들 호출!")
21 # C++ 코드에서는 [out, retval] int* result였지만,
22 # 파이썬에서는 깔끔하게 리턴값(=>으로 받을 수 있게 변환됨
23
24
25 # C++ 구조체(POINT3D) 파라미터 테스트
26 pt = MyMathLib.tagPOINT3D()
27 pt.x = 10
28 pt.y = 20
29 pt.z = 30
30 res_pt = math_server.AddPoint(byref(pt))
31 print(f"  AddPoint(10, 20, 30) = {res_pt}")
32
33 res_add = math_server.Add(100, 50)
34 print(f"  100 + 50 = {res_add}")
35
36 res_sub = math_server.Subtract(100, 50)
37 print(f"  100 - 50 = {res_sub}")
38
39 res_mul = math_server.Multiply(100, 50)
40 print(f"  100 * 50 = {res_mul}")
41
42
43 print("\n4. 통신 종료 (서버 쪽에 5초 뒤 종료 타이머가 작동할 것입니다!)")
44 del math_server
```

Write com_mymath.py

- comtypes.gen에 .tlb를 조회하고 class가 정의됨 (불편..)



```
Server > test_v1.py > ...
1 import os
2 from ctypes import byref
3
4 from comtypes import CLSCTX_LOCAL_SERVER
5 from comtypes.client import CreateObject, GetModule
6
7 print("==== [Unknown 방식 - V1] =====")
8
9 # 1. 런타임에 설명서(.tlb)를 강제로 읽어와서 파이썬 클래스로 번역해야 함
10 tlb_path = os.path.abspath(r"D:\Users\baekjoon\cpp_code\COM_Outofproc\Server\IMyMath.tlb")
11 GetModule(tlb_path)
12
13 # TLB를 읽은 후에야 비로소 생성된 모듈(MyMathLib)을 임포트 가능
14 from comtypes.gen import MyMathLib # noqa: E402
15
16 # 2. 객체 생성 (CLSID 또는 생성된 파이썬 래퍼 클래스를 통해 생성)
17 math_server = CreateObject(MyMathLib.MyMathComponent, clsctx=CLSCTX_LOCAL_SERVER)
18
19 # 3. UDT(사용자 정의 구조체) 세팅 및 함수 호출
20 # 파이썬에서 C++ 구조체를 용내내어 직접 만들어야 함
21 pt = MyMathLib.tagPOINT3D()
22 pt.x = 10
23 pt.y = 20
24 pt.z = 30
25
26 # C++ 함수가 포인터(PPOINT3D*)를 원하므로 byref()로 메모리 주소를 넘김
27 res_pt = math_server.AddPoint(byref(pt))
28 print(f"AddPoint 결과 (구조체+포인터): {res_pt}")
29
30 res_add = math_server.Add(100, 50)
31 print(f"Add 결과: {res_add}")
32
33 del math_server
```

Write com_mymath.py

- CreateObject: CoCreateInstance 호출
 - CreateObject의 반환값이 COM Server
- MyMathLib.tagPOINT3D() 처럼 사용 가능
- Use case
 - math_server.AddPoint(byref(pt))
 - ✓ byref: 참조에 의한 전달(&pt) > 주소를 보냄

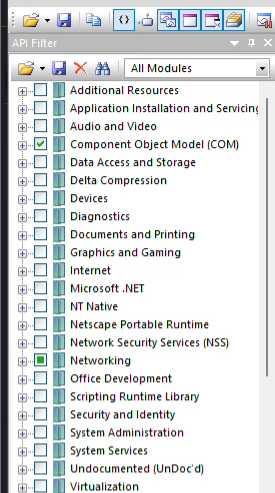
```
Server > test_v1.py > ...
1 import os
2 from ctypes import byref
3
4 from ctypes import CLSCTX_LOCAL_SERVER
5 from ctypes.client import CreateObject, GetModule
6
7 print("==== [Unknown 방식 - V1] =====")
8
9 # 1. 런타임에 설명서(.tlb)를 강제로 읽어와서 파이썬 클래스로 번역해야 함
10 tlb_path = os.path.abspath(r"D:\Users\baekjoon\cpp_code\COM_Outofproc\Server\IMyMath.tlb")
11 GetModule(tlb_path)
12
13 # TLB를 읽은 후에야 비로소 생성된 모듈(MyMathLib)을 임포트 가능
14 from ctypes.gen import MyMathLib # noqa: E402
15
16 # 2. 객체 생성 (CLSID 또는 생성된 파이썬 래퍼 클래스를 통해 생성)
17 math_server = CreateObject(MyMathLib.MyMathComponent, clsctx=CLSCTX_LOCAL_SERVER)
18
19 # 3. UDT(사용자 정의 구조체) 세팅 및 함수 호출
20 # 파이썬에서 C++ 구조체를 용내내어 직접 만들어야 함
21 pt = MyMathLib.tagPOINT3D()
22 pt.x = 10
23 pt.y = 20
24 pt.z = 30
25
26 # C++ 함수가 포인터(PVOID*)를 원하므로 byref()로 메모리 주소를 넘김
27 res_pt = math_server.AddPoint(byref(pt))
28 print(f"AddPoint 결과 (구조체+포인터): {res_pt}")
29
30 res_add = math_server.Add(100, 50)
31 print(f"Add 결과: {res_add}")
32
33 del math_server
```

Run

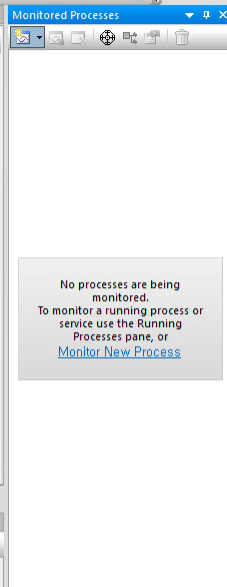
- Step 5. `mathserver.exe`, `com_mymath.py` 동작

```
==> [IUnknown 방식 - V1] ==>  
AddPoint 결과 (구조체+포인터): 60  
Add 결과: 150  
(myenv) PS D:\Users\baekjoon\cpp_code\COM_Outofproc> █
```

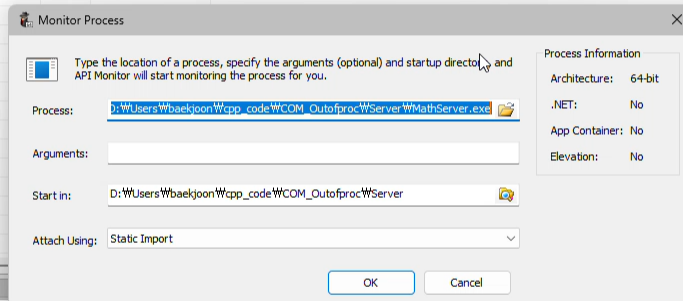
```
==== Out-of-Proc COM Server Start ====  
[+] Server received 1  
[+] Server received 2  
[+] Server received 3  
[+] Server received 4  
[!] Enter Sleep Mode... (Ends in 5 sec)  
g_ShutdownTimerId: 26287
```



Process	PID
ai.exe	14712
AppActions.exe	9228
ApplicationFrameHost...	5108
AppVshNotify.exe	6248
backgroundTaskHost...	14036
backgroundTaskHost...	16264
brave.exe	3976
brave.exe	18268
brave.exe	16032
brave.exe	12392
brave.exe	16904
brave.exe	18164
brave.exe	2164
brave.exe	19244
brave.exe	7608
conhost.exe	16416
CrossDeviceResume.exe	10372

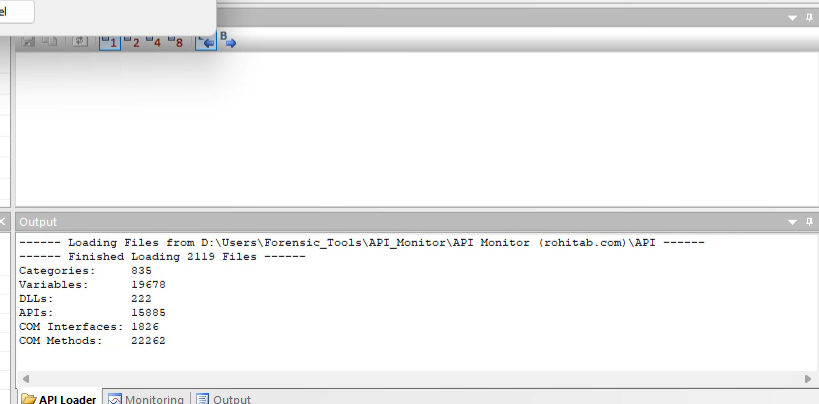


#	Time of Day	Thread	Module	API	Return Value	Error
No processes are being monitored.						



#	Type	Name	Pre-Call Value	Post-Call Value
Parameters				

#	Module	Address	Offset	Location
Call Stack				



diff.

- diff. IUnknown, IDispatch

	IUnknown	IDispatch
함수 탐색 방식	초기바인딩	지연바인딩
데이터 타입 범용성	User Defined Type 가능 다중 포인터 가능 > 파이썬에서 .tlb를 읽어 파이썬 클래스로 번역해야 함 >> GetModule(tlb_path)	가능은 한데 *_*VARIANT 타입에 실어 보내야하며, VT_RECORD를 통해 IRecordInfo를 동봉 해야 함. +. 인자로 하나씩 보내거나 +. 문자열로 보내거나.. 우회 필요
통신(마샬링)	전용 PS.dll 필수(UDT 있으면)	전용 PS.dll 불필요 (oleaut32.dll이라는 범용 PS.dll 사용 가능)
사용처 범용성	스크립트 언어 불가능.. 파이썬 OK (.tlb 필수) > 불편한 사용	스크립트 언어 가능.. 파이썬 OK (.tlb 불필요) > 편안한 사용

VARIANT type

- pvRecord: UDT
- IRecordInfo: UDT에 대한 설명
- VARIANT: 여러 타입이 될 수 있음

```
void PackVariantExample() {  
    VARIANT var;  
    VariantInit(&var); // VARIANT 상자 초기화  
  
    // (가정) 문자열 배열이 생성되어 있다고 칭  
    SAFEARRAY* mySafeArray = NULL;  
  
    // 1. 플래그 세팅 (매크로 V_VT 사용)  
    V_VT(&var) = VT_BSTR | VT_ARRAY | VT_BYREF;  
  
    // 2. 데이터 세팅 (매크로 V_ARRAYREF 사용)  
    // 포인터의 포인터 공간에 주소를 넣음  
    V_ARRAYREF(&var) = &mySafeArray;  
}
```

```
void ParseComplexVariant(VARIANT* pVar) {  
    // 1. 기본 자료형 추출: 하위 비트만 남겨서 알맹이가 문자열인지 확인  
    // V_VT 매크로를 사용하여 안전하게 플래그를 가져옵니다.  
    if ((V_VT(pVar) & VT_TYPEMASK) == VT_BSTR) {  
        printf("[확인] 알맹이는 문자열(BSTR)입니다.\n");  
  
        // 2. 배열 (ARRAY) 속성이 섞여 있는지 확인  
        if (V_VT(pVar) & VT_ARRAY) {  
            printf("[확인] 단일 문자열이 아니라 문자열 배열입니다.\n");  
  
            // 3. 포인터 (BYREF) 속성이 섞여 있는지 확인  
            if (V_VT(pVar) & VT_BYREF) {  
                printf("[확인] 배열의 포인터(주소)로 넘어왔습니다!\n");  
  
                // V_ARRAYREF 매크로를 사용하여 SAFEARRAY** 주소를 획득  
                SAFEARRAY** ppSA = V_ARRAYREF(pVar);  
  
                // 여기서 원본 배열(*ppSA) 직접 조작...  
            }  
        }  
        else {  
            printf("[확인] 일반 배열 복사본으로 넘어왔습니다.\n");  
  
            // V_ARRAY 매크로를 사용하여 SAFEARRAY* 주소를 획득  
            SAFEARRAY* pSA = V_ARRAY(pVar);  
  
            // 배열 파싱 로직...  
        }  
    }  
}
```

```
typedef struct tagVARIANT {  
    union {  
        struct {  
            VARTYPE vt;  
            WORD wReserved1;  
            WORD wReserved2;  
            WORD wReserved3;  
            union {  
                LONGLONG l1Val;  
                LONG lVal;  
                BYTE bVal;  
                SHORT sVal;  
                FLOAT fltVal;  
                DOUBLE dblVal;  
                VARIANT_BOOL boolVal;  
                VARIANT_BOOL __OBsolete__VARIANT_BOOL;  
                SCODE scode;  
                CY cyVal;  
                DATE date;  
                BSTR bstrVal;  
                IUnknown *punkVal;  
                IDispatch *pdispVal;  
                SAFEARRAY *parray;  
                BYTE *pbVal;  
                SHORT *piVal;  
                LONG *plVal;  
                LONGLONG *pllVal;  
                FLOAT *pfltVal;  
                DOUBLE *pdblVal;  
                VARIANT_BOOL *pboolVal;  
                VARIANT_BOOL __OBsolete__VARIANT_BOOL;  
                SCODE *pscode;  
                CY *pcyVal;  
                DATE *pdate;  
                BSTR *pbstrVal;  
                IUnknown **ppunkVal;  
                IDispatch **ppdispVal;  
                SAFEARRAY **pparray;  
                VARIANT *pvarVal;  
                PVOID byref;  
                CHAR cVal;  
                USHORT uiVal;  
                ULONG ulVal;  
                ULONGLONG ul1Val;  
                INT intVal;  
                UINT uintVal;  
                DECIMAL *pdecimal;  
                CHAR *pcVal;  
                USHORT *puVal;  
                ULONG *pulVal;  
                ULONGLONG *pullVal;  
                INT *pintVal;  
                UINT *puintVal;  
            }  
        }  
        struct {  
            PVOID pvRecord;  
            IRecordInfo *pRecInfo;  
        }  
    }  
    __VARIANT_NAME_3;  
} __VARIANT_NAME_2;  
DECIMAL decVal;  
} __VARIANT_NAME_1;  
} VARIANT;
```

IDispatch

- IDispatch 필수 함수

- GetTypeInfoCo

- ✓ TypeInfo(타입)

- GetTypeInfo

- ✓ ITypeInfo 인터페이스

- ✓ ITypeInfo: .tlb를 OS

- GetIDsOfNames

- ✓ 함수의 이름으로

- Invoke

- ✓ 함수 번호로 함수

DispGetIDsOfNames

> 동적으로 적용 가능

DispInvoke 함수(oleauto.h)

인터페이스에 대한 형식 정보가 제공되면 인터페이스에서 멤버 함수를 자동으로 호출합니다.
단일 호출을 사용하여 인터페이스에 대해 **Invoke** 를 구현할 수 있습니다.

구문

C++

```
HRESULT DispInvoke(  
    void        *_this,  
    ITypeInfo    *ptinfo,  
    DISPID      dispidMember,  
    WORD        wFlags,  
    DISPPARAMS *pparams,  
    VARIANT     *pvarResult,  
    EXCEPINFO   *pexcepinfo,  
    UINT        *puArgErr  
);
```

Server_v2 > mathserver.cpp > fx LoadMyTypeInfo()

```
39 class MyMathServer : public IMathV2 {  
73 // IDispatch  
74
```

```
erride {
```

```
, LCID lcid, DISPID *rgDispId) override {
```

```
lags, DISPPARAMS *pDispParams, VARIANT *pVarResult, EX
```

```
t, pExcepInfo, puArgErr);
```

```
120 *result = a * b;  
121 return S_OK;  
122 }
```

```

HRESULT STDMETHODCALLTYPE GetIDsOfNames(REFIID riid, LPOLESTR *rgszNames, UINT cNames, LCID lcid, DISPID *rgDispId) override {
    if (wcscmp(rgszNames[0], L"AnalyzeLogFile") == 0)
    {
        *rgDispId = 1;
    }
    else if (wcscmp(rgszNames[0], L"GetLineCounts") == 0)
    {
        *rgDispId = 2;
    }
    else if (wcscmp(rgszNames[0], L"MaxScanLimit") == 0)
    {
        *rgDispId = 3;
    }
    else
    {
        return DISP_E_UNKNOWNNAME;
    }
    return S_OK;
}

```

```

HRESULT STDMETHODCALLTYPE Invoke(DISPID dispIdMember, REFIID riid, LCID lcid, WORD wFlags, DISPPARAMS *pDispParams,
                                VARIANT *pVarResult, EXCEPINFO *pExcepInfo, UINT *puArgErr) override {
    /*
    DISPID: Member Index
    pDispParams: Parameter List
    pVarResult: Return value
    */
    // [id(1)] AnalyzeLogFile 호출
    if (dispIdMember == 1 && (wFlags & INVOKE_FUNC)) {
        if (pDispParams->cArgs != 2)
            return DISP_E_BADPARAMCOUNT;
        return AnalyzeLogFile(pDispParams->rgvarg[1].bstrVal, pDispParams->rgvarg[0].bstrVal, pVarResult);
    }
    // [id(2)] GetLineCounts 호출
    else if (dispIdMember == 2 && (wFlags & INVOKE_FUNC)) {
        if (pDispParams->cArgs != 2)
            return DISP_E_BADPARAMCOUNT;
        long total = 0, matched = 0;
        HRESULT hr = GetLineCounts(pDispParams->rgvarg[1].bstrVal, pDispParams->rgvarg[0].bstrVal, &total, &matched);
        //$ 파이썬(초기 바인딩)을 위해 C++ 단에서 튜플처리를 대비해 값 우선 세팅(빈손 방지 fallback).
        if (SUCCEEDED(hr) && pVarResult) {
            pVarResult->vt = VT_I4;
            pVarResult->lVal = matched; // 동적 바인딩일 경우 매치된 수만 우선 리턴(total은 보낼 수가 없나?)
        }
        return hr;
    }
    // [id(3)] MaxScanLimit 프로퍼티 읽기/쓰기 호출
    else if (dispIdMember == 3) {
        //$ Getter
        if (wFlags & INVOKE_PROPERTYGET) { // 파이썬: a = scanner.MaxScanLimit
            if (pVarResult) {
                pVarResult->vt = VT_I4;
                return get_MaxScanLimit(&pVarResult->lVal);
            }
        }
        //$ Setter
        else if (wFlags & INVOKE_PROPERTYPUT) { // 파이썬: scanner.MaxScanLimit = 500
            if (pDispParams->cArgs != 1)
                return DISP_E_BADPARAMCOUNT;
            return put_MaxScanLimit(pDispParams->rgvarg[0].lVal);
        }
    }
    return DISP_E_MEMBERNOTFOUND;
}

```

Write com_mymath.py v2.0

- 특징

1. IDispatch는 경로 필요 없이 등록된 Prog ID 만으로 호출 가능

2. .tlb는 파이썬에서는 알 필요 없음 > test_v1.py의 GetModule ❌

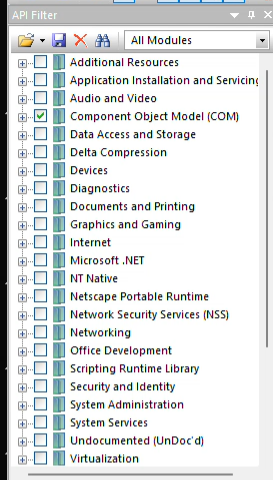
```
Server > test_v1.py > ...
1 import os
2 from ctypes import byref
3
4 from ctypes import CLSCTX_LOCAL_SERVER
5 from ctypes.client import CreateObject, GetModule
6
7 print("==== [Unknown 방식 - V1] =====")
8
9 # 1. 런타임에 설명서(.tlb)를 강제로 읽어와서 파이썬 클래스로 번역해야 함
10 tlb_path = os.path.abspath(r"D:\Users\baekjoon\cpp_code\COM_Outofproc\MyMath.tlb")
11 GetModule(tlb_path)
12
13 # TLB를 읽은 후에야 비로소 생성된 모듈(MyMathLib)을 임포트 가능
14 from ctypes.gen import MyMathLib # noqa: E402
15
16 # 2. 객체 생성 (CLSID 또는 생성된 파이썬 래퍼 클래스를 통해 생성)
17 math_server = CreateObject(MyMathLib.MyMathComponent, clsctx=CLSCTX_LOCAL_SERVER)
18
19 # 3. UDT(사용자 정의 구조체) 세팅 및 함수 호출
20 # 파이썬에서 C++ 구조체를 흉내내어 직접 만들어야 함
21 pt = MyMathLib.tagPOINT3D()
22 pt.x = 10
23 pt.y = 20
24 pt.z = 30
25
26 # C++ 함수가 포인터(PPOINT3D*)를 원하므로 byref()로 메모리 주소를 넘김
27 res_pt = math_server.AddPoint(byref(pt))
28 print(f"AddPoint 결과 (구조체+포인터): {res_pt}")
29
30 res_add = math_server.Add(100, 50)
31 print(f"Add 결과: {res_add}")
32
33 del math_server
```

```
Server_v2 > test_v2.py > ...
1 import win32com.client
2
3 print("\n==== [IDispatch 방식 - V2] =====")
4
5 # 1. 객체 생성
6 # TLB 파일 경로를 적어줄 필요 없이, 윈도우에 등록된 문자열 이름
7 math_server = win32com.client.Dispatch("MyMath.ComponentV2")
8
9 # 2. 함수 호출
10 # 일반 파이썬 함수 부르듯이 인자 3개(기본 자료형)를 그냥 던짐
11 # $ 구조체를 지원하기 너무 복잡해서 인자로 x, y, z를 보냄
12 res_pt = math_server.AddPoint(10, 20, 30)
13 print(f"AddPoint 결과 (기본 자료형): {res_pt}")
14
15 res_add = math_server.Add(100, 50)
16 print(f"Add 결과: {res_add}")
17
18 del math_server
```

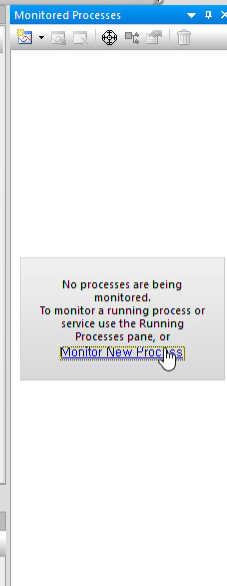
Run v2.0

```
⇒ [IDispatch 방식 - V2] ⇒  
AddPoint 결과 (기본 자료형): 60  
Add 결과: 150  
(myenv) PS D:\Users\baekjoon\cpp_code\COM_Outofproc>
```

```
==== Out-of-Proc COM Server V2 Start (IDispatch) ====  
[!] Enter Sleep Mode... (Ends in 5 sec)
```



Process	PID
agv.exe	13424
ai.exe	14712
AppActions.exe	9228
ApplicationFrameHost...	5108
AppVShNotify.exe	6248
backgroundTaskHost...	14036
backgroundTaskHost...	19552
backgroundTaskHost...	15756
Code.exe	16040
Code.exe	16184
Code.exe	8648
Code.exe	8068
Code.exe	9920
Code.exe	3660
Code.exe	9200
Code.exe	11420
Code.exe	6748



Parameters				
#	Type	Name	Pre-Call Value	Post-Call Value

Call Stack				
#	Module	Address	Offset	Location

#	Time of Day	Thread	Module	API	Return Value	Error

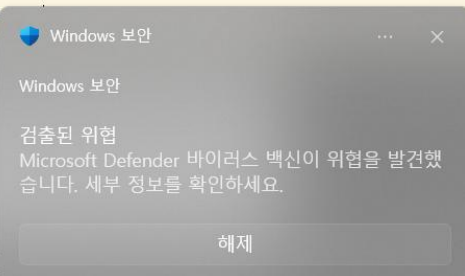
Hex Buffer				

Output				
----- Loading Files from D:\Users\Forensic_Tools\API_Monitor\API Monitor (rohithab.com)\API -----				
----- Finished Loading 2119 Files -----				
Categories:	835			
Variables:	19678			
DLLs:	222			
APIs:	15085			
COM Interfaces:	1826			
COM Methods:	22262			

Issue.

```
D:\Users\baekjoon\cpp_code\COM_Outofproc\Client>.\Client_wait.exe
시스템이 지정된 프로그램을 실행할 수 없습니다.
```

```
D:\Users\baekjoon\cpp_code\COM_Outofproc\Client>
```



- ?? 트로이목마로 판단하고 막음.. 공격자의 명령을 실행한다는데 그런거 없음

Trojan:Win64/CymRan.ACY!MTB

경고 수준: 심각

상태: 적극적

날짜: 2026-05-29 오후 5:20

범주: 트로이

세부 정보: 이 프로그램은 위험하며 공격자의 명령을 실행합니다.

자세한 정보

영향을 받는 항목:

file: D:\Users\baekjoon\cpp_code\COM_Outofproc\Client\WClient_wait.exe

!MTB: 머신러닝/행위 기반 탐지

이 코드가 악성코드와 비슷하게 행동함을 발견한거임 > 기기 허용 필요

- 어째서? Defender 입장에서 수상.. CoCreateInstance: 레지스트리 조회, LocalServer32 조회, 다른 exe 실행
- 서버 코드가 레지스트리 자동 등록, RPC 사용
- 전체적으로 악성 코드의 주요 동작들임

Issue.

```
PS C:\Users\psw01> $math = New-Object -ComObject "MyMath.ComponentV2"  
PS C:\Users\psw01> $math.AddPoint(10,20,30)
```

```
60
```

```
PS C:\Users\psw01>
```

```
0
```

```
PS C:\Users\psw01>
```

```
24A5"
```

```
New-Object
```

```
EGDB_E_CL
```

```
소의 COM
```

```
위치 줄:
```

```
+ $mathV1
```

```
+
```

```
+ Cat
```

```
+ Full
```

```
ctCom
```

```
PS C:\Users\psw01>
```

```
$voice = New-Object -ComObject SAPI.SpVoice
```

```
Write-Host "현재 설정된 목소리: " $voice.Voice.GetDescription()
```

```
$voice.Rate = 1 # 말하는 속도 (-10 ~ 10)
```

```
$voice.Volume = 100 # 볼륨 (0 ~ 100)
```

```
$textToSpeak = "Hello world!!"
```

```
$voice.Speak($textToSpeak)
```

```
PS C:\Users\psw01> $type = [Type]::GetTypeFromCLSID("08564873-A821-4119-B5EE-79DCA34324A5")
```

```
PS C:\Users\psw01> $mathV1 = [System.Activator]::CreateInstance($type)
```

```
PS C:\Users\psw01> $mathV1.Add(100, 50)
```

[System.__ComObject]에 이름이 'Add'인 메서드가 없으므로 메서드를 호출하지 못했습니다.

위치 줄:1 문자:1

```
+ $mathV1.Add(100, 50)
```

```
+ ~~~~~
```

```
+ CategoryInfo          : InvalidOperation: (:) [], RuntimeException
```

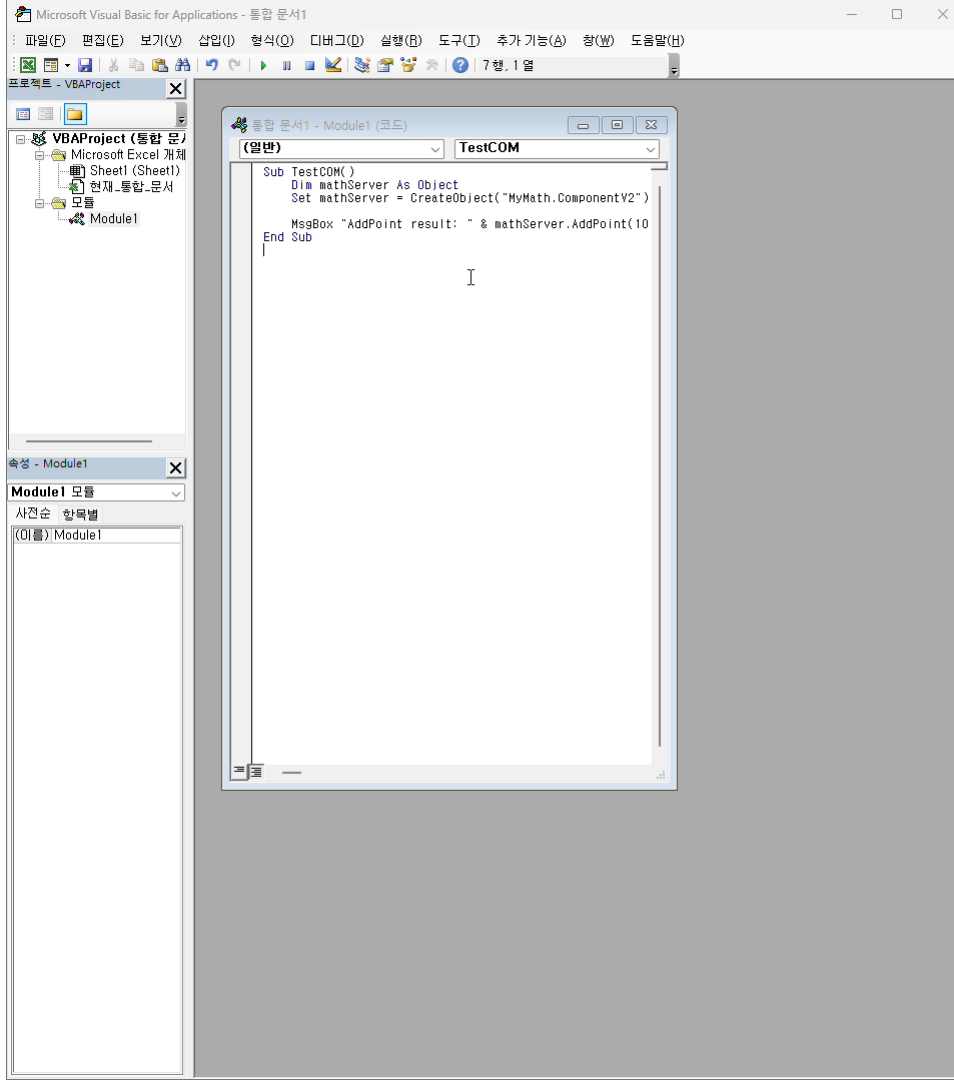
```
+ FullyQualifiedErrorId : MethodNotFound
```


Issue.

```
D:\Users\baekjoon\cpp_code\COM_Outofproc\Server_v2>cscript //nologo ..\Server\test_v1.js
==== V1 Failure Test ====
[!] Expected Error Occurred.
Error: 자동화 서버는 개체를 작성할 수 없습니다.
Reason: V1 is IUnknown only.

D:\Users\baekjoon\cpp_code\COM_Outofproc\Server_v2>cscript //nologo test.js
==== JScript (WSH) V2(IDispatch) Success Test ====
[+] Server Connected Successfully!
    AddPoint(10, 20, 30) = 60
    Add(100, 50) = 150

==== Test Completed ====
```



VBScript로 작성

- > 악용사례가 많다고 함
- >> 여러가지 해볼 수 있음..
- >> COM을 많이 사용한다 함

- >>> 나중에 알아보까?



GeminiWor...

Seminar

숫몸자락자...



3-1

이 사진에
대해 자세...



app.honi

Thank you

Step 2. Edit mathserver.cpp

- FLOW
- **IMyMath.idl** 수정
- coclass를 추가하지 않으면 파이썬이 인터페이스의 함수는 파악 가능, 근데 CLSID를 사용자가 하드코딩 해줘야하는 문제 발생
- coclass를 추가하면? →
- + extern const CLSID CLSID_MyMathComponent; 가능
- **regsvr32 /s MathServerPS.dll** 동작 후?
- 경로: HKCR\Interface\{A1B2C3D4...} > **지금 UUID도 이상함 (원래 AB로 끝나는데)**

컴퓨터\HKEY_CLASSES_ROOT\Interface\{A1B2C3D4-E5F6-7890-ABCD-EF1234567890}\ProxyStubClsid32

	이름	종류	데이터
ProxyStubClsid32	(기본값)	REG_SZ	<u>{95E15D0A-66E6-93D9-C53C-76E6219D3341}</u>

- ProxyStubClsid32라는 키가 생긴 것을 볼 수 있는데 CLSID 확인..

- 조회 컴퓨터\HKEY_CLASSES_ROOT\CLSID\{95E15D0A-66E6-93D9-C53C-76E6219D3341}\InProcServer32

	이름	종류	데이터
InProcServer32	(기본값)	REG_SZ	C:\Windows\System32\OneCoreUAPCommonProxyStub.dll
Threadin...		REG_SZ	Both

Step 2. Edit mathserver.cpp

- FLOW

- 이런!! Win 기본 프로시시스템DLL이 매핑됨

- 왜? 너무 간단해서

- ✓ 사용자 정의 자료형 추가 (e.g. typedef)

- 어디에? IMyMath.idl

- + UUID가 이상했던 이유? MathServerPS.dll을 등록해야 하는데 MathServer.dll을 등록함..

컴퓨터\HKEY_CLASSES_ROOT\Interface\{A1B2C3D4-E5F6-4A1B-8C9D-0123456789AB}\ProxyStubClsid32			
> [A1AD48B0-C496-4211-9782-9F086671]	이름	종류	데이터
▼ [A1B2C3D4-E5F6-4A1B-8C9D-0123456789AB]	ab (기본값)	REG_SZ	[A1B2C3D4-E5F6-4A1B-8C9D-0123456789AB]
NumMethods			
ProxyStubClsid32			

- ProxyStubClsid32라는 키가 생긴 것을 볼 수 있는데 Clsid에 집중

- 조회 컴퓨터\HKEY_CLASSES_ROOT\CLSID\{A1B2C3D4-E5F6-4A1B-8C9D-0123456789AB}\InProcServer32

> [A1B1922F-B626-4A4F-9F2B-4A6926D1]	이름	종류	데이터
▼ [A1B2C3D4-E5F6-4A1B-8C9D-0123456789AB]	ab (기본값)	REG_SZ	D:\users\baekjoon\cpp_code\COM_Outofproc\Server\MathServerPS.dll
InProcServer32	ab Threading...	REG_SZ	Both

Step 2. Edit mathserver.cpp

- 실습: MathServer.exe 수행

```
PS D:\Users\baekjoon\cpp_code\COM_Outofproc\Server> .\MathServer.exe
==== Out-of-Proc COM Server Start ====
|
```

백그라운드 프로세스 (1)				
 MathServer.exe	0%	1.0MB	0MB/s	0Mbps

- client_outofproc.cpp 컴파일 및 수행

```
D:\Users\baekjoon\cpp_code\COM_Outofproc\Client>cl /utf-8 /EHsc /I*..\Server" client_outofproc.cpp *..\Server\IMyMath.i.
c* /link ole32.lib /out:Client.exe
Microsoft (R) C/C++ 최적화 컴파일러 버전 19.44.35208(x64)
Copyright (c) Microsoft Corporation. All rights reserved.

client_outofproc.cpp
코드를 생성하고 있습니다...
컴파일하고 있습니다...
IMyMath.i.c
코드를 생성하고 있습니다...
Microsoft (R) Incremental Linker Version 14.44.35208.0
Copyright (C) Microsoft Corporation. All rights reserved.

/out:client_outofproc.exe
ole32.lib
/out:Client.exe
client_outofproc.obj
IMyMath.i.obj

D:\Users\baekjoon\cpp_code\COM_Outofproc\Client>
```

- 전체적인 틀은 갖추었고, 수행 (문제 상황 존재)

```
PS D:\Users\baekjoon\cpp_code\COM_Outofproc\Server> .\MathServer.exe
==== Out-of-Proc COM Server Start ====
|
```

- 클라이언트 수행 후 바로 종료하는 문제

```
PS D:\Users\baekjoon\cpp_code\COM_Outofproc\Client> .\Client.exe
==== COM Client Start ====
Success! Found and connected to MathServer.
10 + 5 = 15
10 - 5 = 5
10 * 5 = 50
AddPoint(10, 20, 30) = 60
==== COM Client End ====

PS D:\Users\baekjoon\cpp_code\COM_Outofproc\Server> .\MathServer.exe
==== Out-of-Proc COM Server Start ====
[+] COM Ref count is 0. (COM Terminated)
```


Step 2. Edit mathserver.cpp

- i.e.) 변동 필요 (Each of *client-side*, *server-side*)

- **client side**

- 연결 유지(세션 관리): main에서 계산하고 바로 Release()호출 X
- TODO: pMath 포인터를 전역 변수나 메인 클래스 멤버 변수로 쥐고 있게 하기 (일반적인 App의 흐름을 유지)
- 사용자가 App을 쓰는 동안 종종 포인터로 서버에 요청 수행하기
- 사용자의 종료 요청을 통해 Release() 호출 수행하기

- **server side**

- 즉시 종료 방지 필요
- 카운트가 0이 되면, 대기시간(1분) 후 그래도 참조가 없으면 종료하게 수정
- 이벤트 기반 통신:

- 문제 또 발생 (MathServer.exe /RegisterServer를 수행하지 않고 시도했는데 됨..)

- 아래 사진은 /RegisterServer 수행하고 찍은거

컴퓨터\HKEY_CLASSES_ROOT\CLSID\{C5D6E7F8-1A2B-3C4D-5E6F-7A8B9C0D1E2F}\LocalServer32

> [C5C5C5F1-3ABC-11D6-B25B-00C04F...]	이름	종류	데이터
▼ [C5D6E7F8-1A2B-3C4D-5E6F-7A8B9C...]	ab (기본값)	REG_SZ	D:\Users\baekjoon\cpp_code\COM_Outofproc\Server\MathServer.exe
InprocServer32			
LocalServer32			
[C5D6E7F8-1A2B-3C4D-5E6F-7A8B9C...]			

- 찾아보니 > MathServer.exe를 먼저 수행하고 클라이언트를 연결했는데
- 먼저 수행한다는 의미는 CoRegisterClassObject가 main에서 호출되어 COM Runtime에게 내가 수행가능함을 알렸기 때문에 가능한거였고..
- /RegisterServer를 수행하면 MathServer.exe가 죽어있어도 Win이 실행시킨다고 함.
- Let's check it out!!